SQL Assignment

Q1) Write an SQL query to report the managers with at least five direct reports.

Return the result table in any order. The query result format is in the following example.

Input:

Employee table:

id	name	department	managerId
101	John	A	None
102	Dan	A	101
103	James	A	101
104	Amy	A	101
105	Anne	A	101
106	Ron	B	101

Output:

name
John

Create Table `Employee`:

Query | Query History

```sql
CREATE TABLE Employee(
        id INT PRIMARY KEY,
        name VARCHAR(50),
        department VARCHAR(50),
        managerId INT);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 124 msec.
```

Populate with input data:

Query | Query History

```
INSERT INTO Employee VALUES
        (101,'Jhon','A',NULL),
        (102,'Dan','A',101),
        (103,'James','A',101),
        (104,'Amy','A',101),
        (105,'Anne','A',101),
        (106,'Ron','B',101);
```

Data Output | Messages | Notifications

```
INSERT 0 6

Query returned successfully in 56 msec.
```

Query:

Query | Query History

```
SELECT m.name
FROM Employee e
JOIN Employee m ON e.managerId=m.id
GROUP BY m.id,m.name
HAVING COUNT(e.id)>=5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	name character varying (50)
1	Jhon

Q2) Write an SQL query to report the nth highest salary from the Employee table. If there is no nth highest salary, the query should report null.

The query result format is in the following example.

Input:

Employee table:

id	salary
1	100
2	200

id	salary
3	300

n = 2

Output:

getNthHighestSalary(2)
200

Input:

Employee table:

id	salary
1	100

n = 2

Output:

getNthHighestSalary(2)
null

Query:

Query | Query History

```
SELECT (
    SELECT salary
    FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
        FROM Employee) AS ranked
    WHERE salary_rank=2) AS getNthHighestSalary;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	getnthhighestsalary integer
1	400

Q3) Write an SQL query to find the people who have the most friends and the most friends number.

The test cases are generated so that only one person has the most friends.
The query result format is in the following example.

Input:

RequestAccepted table:

requester_id	accepter_id	accept_date
1	2	2016/06/03
1	3	2016/06/08
2	3	2016/06/08
3	4	2016/06/09

Output:

id	num
3	3

Create table `RequestAccepted` :

Query | Query History

```
CREATE TABLE RequestAccepted(
    requester_id INT,
    accepter_id INT,
    accept_date DATE,
    PRIMARY KEY(requester_id,accepter_id)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 76 msec.
```

Populate with input data:

Query | Query History

```
INSERT INTO RequestAccepted (requester_id, accepter_id, accept_date)
VALUES
    (1, 2, '2016-06-03'),
    (1, 3, '2016-06-08'),
    (2, 3, '2016-06-08'),
    (3, 4, '2016-06-09');
```

Data Output | Messages | Notifications

```
INSERT 0 4

Query returned successfully in 68 msec.
```

Query:

Query | Query History

```sql
SELECT id, COUNT(*) AS num
FROM (
    SELECT requester_id AS id
    FROM RequestAccepted
    UNION ALL
    SELECT accepter_id AS id
    FROM RequestAccepted
) AS friends
GROUP BY id
ORDER BY num DESC
LIMIT 1;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	id integer	num bigint
1	3	3

Q4) Write an SQL query to swap the seat id of every two consecutive students. If the number of students is odd, the id of the last student is not swapped.

Column Name	Type
id	int
name	varchar

`id` is the primary key column for this table.
Each row of this table indicates the name and the ID of a student.
`id` is a continuous increment.

Return the result table ordered by id in ascending order.
The query result format is in the following example.

Input:

Seat table:

id	student
1	Abbot
2	Doris

id	student
3	Emerson
4	Green
5	Jeames

Output:

id	student
1	Doris
2	Abbot
3	Green
4	Emerson
5	Jeames

Create table `Seat`:

Query | Query History

```sql
CREATE TABLE Seat (
    id INT PRIMARY KEY,
    student VARCHAR(50)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 89 msec.
```

Populate with input data

Query | Query History

```sql
INSERT INTO Seat (id, student)
VALUES
    (1, 'Abbot'),
    (2, 'Doris'),
    (3, 'Emerson'),
    (4, 'Green'),
    (5, 'Jeames');
```

Data Output | Messages | Notifications

```
INSERT 0 5

Query returned successfully in 95 msec.
```

Query:

Query | Query History

```sql
SELECT CASE
        WHEN id % 2 = 1 AND id < (SELECT MAX(id) FROM Seat) THEN id + 1
        WHEN id % 2 = 0 THEN id - 1
        ELSE id
    END AS id,
    student
FROM Seat
ORDER BY id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	id integer	student character varying (50)
1	1	Doris
2	2	Abbot
3	3	Green
4	4	Emerson
5	5	Jeames

Q5) Write an SQL query to report the customer ids from the Customer table that bought all the products in the Product table.

Return the result table in any order.

The query result format is in the following example.

Input:

Customer table:

customer_id	product_key
1	5
2	6
3	5
3	6
1	6

Product table:

product_key
5
6

Output:

customer_id
1
3

Query:

Query | Query History

```sql
SELECT customer_id
FROM Customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) = (
    SELECT COUNT(*)
    FROM Product
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	customer_id integer
1	1
2	3

Q6) Write an SQL query to find for each user, the join date and the number of orders they made as a buyer in 2019.

Return the result table in any order.

The query result format is in the following example.

Input:

Users table:

user_id	join_date	favorite_brand
1	2018-01-01	Lenovo
2	2018-02-09	Samsung
3	2018-01-19	LG
4	2018-05-21	HP

Orders table:

order_id	order_date	item_id	buyer_id	seller_id
1	2019-08-01	4	1	2
2	2018-08-02	2	1	3
3	2019-08-03	3	2	3
4	2018-08-04	1	4	2
5	2018-08-04	1	3	4
6	2019-08-05	2	2	4

Items table:

item_id	item_brand
1	Samsung
2	Lenovo
3	LG
4	HP

Output:

buyer_id	join_date	orders_in_2019
1	2018-01-01	1
2	2018-02-09	2
3	2018-01-19	0
4	2018-05-21	0

Query:

Query | Query History

```sql
SELECT
    u.user_id AS buyer_id,
    u.join_date,
    COUNT(o.order_id) AS orders_in_2019
FROM Users u
LEFT JOIN Orders o
    ON u.user_id = o.buyer_id
    AND EXTRACT(YEAR FROM o.order_date) = 2019
GROUP BY u.user_id, u.join_date;
```

Q7) Write an SQL query to reports for every date within at most 90 days from today, the number of users that logged in for the first time on that date. Assume today is 2019-06-30.

Return the result table in any order.

The query result format is in the following example.

Input:

Traffic table:

user_id	activity	activity_date
1	login	2019-05-01
1	homepage	2019-05-01
1	logout	2019-05-01
2	login	2019-06-21
2	logout	2019-06-21
3	login	2019-01-01
3	jobs	2019-01-01
3	logout	2019-01-01
4	login	2019-06-21
4	groups	2019-06-21
4	logout	2019-06-21
5	login	2019-03-01
5	logout	2019-03-01
5	login	2019-06-21
5	logout	2019-06-21

Output:

login_date	user_count
2019-05-01	1
2019-06-21	2

Query:

Query | Query History

```sql
WITH first_logins AS (
    SELECT user_id, MIN(activity_date) AS login_date
    FROM Traffic
    WHERE activity = 'login'
    GROUP BY user_id)
SELECT login_date,COUNT(*) AS user_count
FROM first_logins
WHERE login_date BETWEEN DATE '2019-06-30' - 90 AND DATE '2019-06-30'
GROUP BY login_date;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	login_date date	user_count bigint
1	2019-05-01	1
2	2019-06-21	2

Q8) Write an SQL query to find the prices of all products on 2019-08-16. Assume the price of all products before any change is 10.

Return the result table in any order.
The query result format is in the following example.

Input:

Products table:

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17
3	20	2019-08-18

Output:

product_id	price
2	50
1	35
3	10

Query:

Query | Query History

```sql
SELECT p.product_id,
COALESCE(
    (SELECT p2.new_price
    FROM Products AS p2
    WHERE p2.product_id=p.product_id
    AND p2.change_date<="2019-08-16"
    ORDER BY p2.change_date DESC
    LIMIT 1),10) AS price
FROM Products p
GROUP BY p.product_id;
```

Q9) Write an SQL query to find for each month and country: the number of approved transactions and their total amount, the number of chargebacks, and their total amount

Chargebacks contains basic information regarding incoming chargebacks from some transactions placed in Transactions table.
`trans_id` is a foreign key to the id column of Transactions table. Each chargeback corresponds to a transaction made previously even if they were not approved.

Note: In your query, given the month and country, ignore rows with all zeros.
Return the result table in any order.

The query result format is in the following example.

Input:

Transactions table:

id	country	state	amount	trans_date
101	US	approved	1000	2019-05-18
102	US	declined	2000	2019-05-19
103	US	approved	3000	2019-06-10
104	US	declined	4000	2019-06-13
105	US	approved	5000	2019-06-15

Chargebacks table:

trans_id	trans_date
102	2019-05-29
101	2019-06-30
105	2019-09-18

Output:

month	country	approved_count	approved_amount	chargeback_count	chargeback_amount
2019-05	US	1	1000	1	2000
2019-06	US	2	8000	1	1000
2019-09	US	0	0	1	5000

Query | Query History

```sql
WITH summary AS (
    SELECT TO_CHAR(trans_date, 'YYYY-MM') AS month,country,
    COUNT(*) FILTER (WHERE state = 'approved') AS approved_count,
    SUM(amount) FILTER (WHERE state = 'approved') AS approved_amount,0 AS chargeback_count,0 AS chargeback_amount
    FROM Transactions
    GROUP BY month,country
    UNION ALL
    SELECT TO_CHAR(c.trans_date, 'YYYY-MM'),t.country,0,0,COUNT(*),SUM(t.amount)
    FROM Chargebacks c
    JOIN Transactions t ON c.trans_id = t.id
    GROUP BY 1, 2
)
SELECT
    month,country,SUM(approved_count) AS approved_count,SUM(COALESCE(approved_amount, 0)) AS approved_amount,
    SUM(chargeback_count) AS chargeback_count,SUM(chargeback_amount) AS chargeback_amount
FROM summary
GROUP BY month, country;
```

Q10) Write an SQL query that selects the team_id, team_name and num_points of each

team in the tournament after all described matches.

You would like to compute the scores of all teams after all matches. Points are awarded as follows: A team
receives three points if they win a match (i.e., Scored more goals than the opponent team). A team receives
one point if they draw a match (i.e., Scored the same number of goals as the opponent team). A team receives
no points if they lose a match (i.e., Scored fewer goals than the opponent team).

Return the result table ordered by `num_points` in decreasing order. In case of a tie, order the records by `team_id` in increasing order.
The query result format is in the following example.

Input:

Teams table:

team_id	team_name
10	Leetcode FC
20	NewYork FC
30	Atlanta FC
40	Chicago FC
50	Toronto FC

Matches table:

match_id	host_team	guest_team	host_goals	guest_goals
1	10	20	3	0
2	30	10	2	2
3	10	50	5	1
4	20	30	1	0
5	50	30	1	0

Output:

team_id	team_name	num_points
10	Leetcode FC	7
20	NewYork FC	3
50	Toronto FC	3
30	Atlanta FC	1

team_id	team_name	num_points
40	Chicago FC	0

Query | Query History

```sql
SELECT t.team_id,t.team_name,
    SUM(
        CASE
            WHEN t.team_id = m.host_team AND m.host_goals > m.guest_goals THEN 3
            WHEN t.team_id = m.guest_team AND m.guest_goals > m.host_goals THEN 3
            WHEN m.host_goals = m.guest_goals THEN 1
            ELSE 0
        END
    ) AS num_points
FROM Teams t
LEFT JOIN Matches m ON t.team_id IN (m.host_team, m.guest_team)
GROUP BY t.team_id, t.team_name
ORDER BY num_points DESC, t.team_id;
```